

DeepFake Detection - Documentation

Pietro De Angeli

August 25, 2025

Contents

1	Introduction	2
2	Project Structure	2
2.1	FF++	3
2.2	dataset	3
2.3	models	3
2.4	src	4
2.4.1	classifier	4
2.4.2	data_preprocessing	4
2.4.3	tools	4
2.5	Tools	4
2.5.1	UI	5
2.6	test	5
3	Pipeline	5
3.1	Face Detection	5
3.2	Motion Vector Extraction	6
3.3	Feature Computation	6
3.4	Classification	7
4	Training the Model	7
4.1	Augmentation	8
4.2	Evaluation	8
5	Tools & Technologies	9
5.1	Conda	9
5.2	Libraries	9
5.3	RunPod	9

6	Results	10
7	Future Works	10

1 Introduction

This project, part of the “Signal, Image and Video Processing” course, implements the paper *"Efficient Temporally-Aware DeepFake Detection using H.264 Motion Vectors"*. The goal is to distinguish between DeepFake and real videos using **motion vectors** (MVs), a lightweight alternative to **Optical Flow** (OF).

MVs can be extracted directly from the H.264 video encoding (if supported), without further computation. Only motion vectors inside face regions are used, which are detected using the pretrained **YuNet** face detector (a lighter alternative to MTCNN used in the original paper).

The architecture consists of two identical MobileNetV3 networks, slightly modified, where one processes RGB images and the other MV+IM tensors. Their outputs are then combined to produce a final binary classification.

The paper criticizes traditional methods for only using per-frame predictions without considering temporal consistency. This solution leverages such temporal information by comparing how content moves across frames.

Unlike OF, which operates at the pixel level, MVs describe how macroblocks (e.g., 16×16 pixels) move across frames. While less precise, MVs are significantly faster to obtain. Their goal was to find a lighter version of optical flow that could be used online and on low-power devices; inference is efficient and bottlenecked mostly by face detection.

2 Project Structure

```
|- dataset/
  |- video_1/
  |- ...
  |- video_N/
|FF++
|- models/
|- src/
  |- classifier/
  |- data_preprocessing/
  |- tools/
  |- UI
|- test/
```

2.1 FF++

The dataset used is available at Kaggle - FaceForensics. This is a much smaller subset of the original FaceForensics++ dataset. It was chosen because it is easier and less computationally expensive to train, making it suitable for developing a Proof of Concept.

The subset consists of two directories:

- *real*: containing 200 original videos;
- *fake*: containing 200 DeepFake videos.

2.2 dataset

The dataset described above has been preprocessed and restructured into subdirectories, as explained in Section 2.4.2. Each video is stored in its own directory containing the following files:

- **meta.json**: contains the original path of the video, the label (*real* or *fake*), and the number of frames extracted. Typically 100, although sometimes fewer.
- **tensors.pt**: contains the stack of **motion_vector_x**, **motion_vector_y**, and **information_mask** for each frame.
- **faces/**: contains the cropped images of the detected faces.

A global file **manifest.json** is also created, containing two objects: **train** and **test**. Each entry specifies the directory of the video and its corresponding label. The split ratio is 80% for training and 20% for testing.

The manifest file is generated by a script described in Section 2.4.2.

2.3 models

- **YuNet**: a pretrained face detector that outputs bounding boxes and confidence scores. It is the bottleneck of our system and is used to extract faces from 100 random and unique frames of each video.
- **Classifier**: based on **MobileNetV3-Small** with pretrained weights. It implements a two-stream architecture that processes both RGB frames and MV+IM features, outputting the prediction of whether a video is fake.

The paper suggests modifying the base network by adapting the **input** (3 channels, 224x224 images) and the **output** (binary classification). In addition, we introduced a parameterized weighted average to combine the two streams. Unlike our implementation, the original paper uses a fixed, non-learnable averaging.

2.4 src

This directory contains the Python source code, organized into submodules.

2.4.1 classifier

Contains the classifier implementation (Section 2.3).

- `MobileNetV3.py`: downloads the pretrained model and adds additional layers.
- `network.py`: defines the two-stream architecture.
- `training.py`: training procedure (Section 4).
- `evaluation.py`: evaluates model accuracy.

2.4.2 data_preprocessing

Handles preprocessing, including train/test split.

`create_dataset.py` processes each video (pipeline in Section 3), creating the structured dataset. `split_dataset()` generates `manifest.json`, splitting into train (80%) and test (20%).

2.4.3 tools

Contains utility modules:

- `face_detection.py`
- `motion_vectors.py`
- `feature_computation.py`
- `tools.py`

2.5 Tools

Provides helper functions:

- `get_dir_videos()`: returns list of all `.mp4` files in a directory.
- `pipeline()`: runs the entire inference pipeline.

Explained in Section 3.

2.5.1 UI

Contains a simple user interface:

- **Upload Page:** drag-and-drop or browse a video. Clicking "Detect" starts the analysis.
- **Progress Page:** shows a waiting screen during computation.
- **Result Page:** shows final results, execution time for each step, and a button to open the temporary directory with extracted faces.

2.6 test

Contains unit tests for the `src` functions (`pytest`). Checks:

- Dimensions and datatypes of tensors.
- Correct 80/20 train-test split in `split_dataset()`.
- Accuracy of YuNet bounding boxes.
- Reproducibility of training and evaluation with fixed random seed.

3 Pipeline

The pipeline is described in the following sections. A complete implementation of the inference pipeline is available in the file `tools.py` (see Section 2.4.3), through the function `pipeline()`.

3.1 Face Detection

The original paper suggests using MTCNN, but YuNet was chosen instead for its efficiency. Two main classes are defined:

- **FaceBox:** represents a square bounding box (x, y, side).
- **Face:** represents a bounding box and the corresponding 224×224 BGR face image.

The function `initialize_detector()` loads the YuNet model. The function `face_frame_extractor()` returns the largest detected face (if any), cropping and resizing it to 224×224 pixels while centering the bounding box.

Frames can be extracted using two different methods:

- `random_frame_extractor()`: samples random timestamps, decodes a short sequence of frames, and selects one.
- `unique_frame_extractor()`: uses reservoir sampling to guarantee uniform coverage over the entire video.

The wrapper `extract_frames_with_faces()` selects between the two approaches based on the `unique_frames` flag. These functions return up to 100 tuples of the form (frame, face). To enable motion vector extraction, the video is opened with the following option:

```
stream.codec_context.options = {"flags2": "+export_mvs"}
```

3.2 Motion Vector Extraction

The class `MotionVector` stores the following attributes:

- `dst_x, dst_y`: current block location
- `motion_x, motion_y`: displacement values
- `motion_scale`: scaling factor

The method `is_in_face()` checks whether a motion vector falls inside a detected face region.

The function `extract_motion_vectors_and_im()` processes each frame–face pair and returns a list of tuples: (MV_x, MV_y, IM). It handles different frame types:

- I-frames (all-zero motion vectors),
- P/B-frames containing valid motion vectors,
- frames where motion vectors may be missing.

Each motion vector is mapped to macroblocks, and intra-coded blocks are identified using the **Information Mask (IM)**: 0 = inter-coded (with MV), 1 = intra-coded. All outputs are resized to 224×224 using appropriate interpolation.

3.3 Feature Computation

The function `compute_features_frame()` extracts per-frame features as follows:

- Normalizes motion vectors over inter-coded regions (IM == 0).
- Stacks MV_x, MV_y, and IM into a 3-channel feature map.

Then, the function `compute_features_video_tensor()` aggregates the per-frame features into a single video tensor with shape (frames, height, width, 3).

3.4 Classification

Finally, the faces extracted during the Face Detection phase and the tensors containing motion vectors are fed into the network. The classifier is restored from the `models` directory (see Section 2.3), and all inputs are passed through the network to produce the final prediction.

4 Training the Model

The training was performed on 280 videos, with testing carried out on 120 videos (following the 80/20 split defined in `manifest.json`). The setup consisted of 30 epochs; however, due to the early stopping strategy (implemented using K-fold validation), the training stopped after 18 epochs. It should be noted that, given the small size of the training set, we did not expect high accuracy or strong generalization.

During training, the training error decreased approximately linearly, while the validation loss followed a "concave" trend, reaching its minimum at epoch 12. This is the epoch at which the best model was saved.

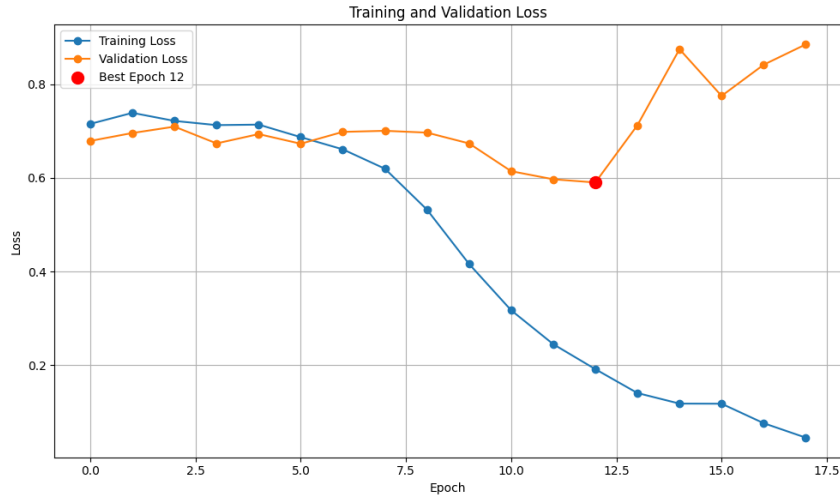


Figure 1: Training and validation loss curves

Two training experiments were carried out:

1. **Without pretraining:** starting from random weights and without augmentation, the model achieved an accuracy of approximately 41%.
2. **With pretraining:** starting from a pretrained model, data augmentation was applied. For the first 5 epochs, only the newly added layers were trained while keeping

the backbone frozen. Afterwards, all weights were unfrozen and fine-tuned. This approach yielded an accuracy of approximately 64%.

4.1 Augmentation

As suggested in the reference paper, the training set was augmented. The augmentation was applied *on the fly*, i.e., only during training and under a certain probability. The library `albumentations` was used to apply the following transformations:

- **RGB distortion**
- **Random brightness and contrast adjustments**
- **Random horizontal/vertical flips:** also the motion vectors are flipped

It has not been used classical techniques such as Gaussian noise, Gaussian blur... because the paper states they reduce the accuracy.

4.2 Evaluation

The evaluation was performed on the 120 test videos, yielding an accuracy of approximately 64%.

The confusion matrix is shown in Figure 2.

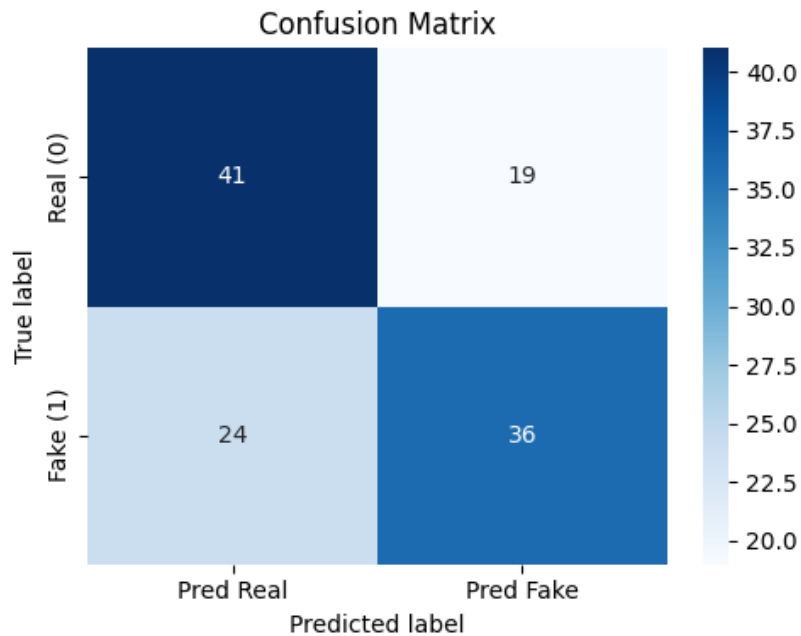


Figure 2: Confusion matrix of the classifier.

From the results we observe:

- Among the real samples (class 0), 41 were correctly classified as real, while 19 were misclassified as fake.
- Among the fake samples (class 1), 36 were correctly classified as fake, while 24 were misclassified as real.

5 Tools & Technologies

5.1 Conda

To set up the project, a dedicated `conda` environment was created with `Python==3.10`. A `setup.py` file was provided to automatically install the required dependencies by running `pip install .` from the main directory.

5.2 Libraries

The main external libraries used are:

- `av` (15.0.0): used to extract motion vectors from videos.
- `albumentations`: applied during training for data augmentation, helping mitigate overfitting.
- `opencv`: used for image and video processing.
- `pytest`: used for unit testing.
- `torch`: used for training the neural network and for the face detection module.

5.3 RunPod

Since training on a local machine was too slow, a cloud platform was used. Specifically, training was performed on a *Pod* hosted on RunPod. Using an `ssh` connection, the preprocessed dataset and the source code (from GitHub) were uploaded, the environment was set up via the `setup.py` file, and the training was executed remotely.

The Pod image came with PyTorch and CUDA pre-installed, and the training was run on an NVIDIA RTX 4090 GPU.

6 Results

The main objective of the project was not to obtain a high-performing network, but rather to reproduce the pipeline proposed in the paper, implement the preprocessing of videos, and set up a training framework for the model. Achieving competitive final results would require significantly more computational resources.

The true goal of this work was to understand and implement the video pre-processing pipeline independently of the AI component. From this perspective, the project can be considered successful, and I am satisfied with the outcome.

7 Future Works

Since the results obtained are not particularly meaningful in terms of accuracy, a natural extension would be to train the model on the full FaceForensics dataset, leveraging more extensive computational resources on RunPod or similar platforms.

Another limitation concerns the use of YuNet. Although it is more efficient than MTCNN, it remains the bottleneck of the pipeline. Future work could therefore explore even lighter models, or potentially solutions that avoid AI-based face detection altogether.